

Guia do Desenvolvedor

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [developers-guide.pt-br.pdf](#)

Executando o CodeBuild Localmente

Você pode executar builds do AWS CodeBuild localmente usando o [agente local do CodeBuild](#). Isso é útil para testar alterações no buildspec sem precisar enviar ao repositório remoto.

Pré-requisitos

- Docker instalado e em execução
- O script `codebuild_build.sh`:

Uso Básico

1. Configuração
2. Baixe o script local do CodeBuild e torne-o executável.
3. Envie o `GH_TOKEN` (Token de Acesso Pessoal do GitHub — PAT) como variável de ambiente para um arquivo `./.env`

```
if [ ! -f codebuild_build.sh ]; then
  curl -O https://raw.githubusercontent.com/aws/aws-codebuild-docker-images/master/local_bui
lds/codebuild_build.sh && chmod +x codebuild_build.sh;
fi;
echo "GH_TOKEN=${GH_TOKEN:-ghp_notset}" > "./.env";
```

1. Iteração
2. *Opcionalmente, edite o valor `buildspec-override` no workflow do GitHub*
`.github/workflows/codebuild.yml`

3. Atualize o `./buildspec.yml` com base no conteúdo do workflow para um arquivo local
4. Execute o build do AWS CodeBuild localmente com imagens baseadas na arquitetura da máquina

```
cat .github/workflows/codebuild.yml \  
  | uvx yq -r '.jobs.build.steps[] | select(.id == "codebuild") | .with["buildspec-overrid  
e"]' \  
  > buildspec.yml  
./codebuild_build.sh \  
  -i "public.ecr.aws/codebuild/amazonlinux-$([ "$arch" = "arm64" -o "$arch" = "aarch64"  
] && echo "aarch64" || echo "x86_64")-standard:$([ "$arch" = "arm64" -o "$arch" = "aarch  
64" ] && echo "3.0" || echo "5.0")" \  
  -a "./.codebuild/artifacts/" \  
  -l "public.ecr.aws/codebuild/local-builds:$([ "$arch" = "arm64" -o "$arch" = "aarch64"  
] && echo "aarch64" || echo "latest")" \  
  -c \  
  -e "./.env"
```

Todas as Opções do Script

Flag	Obrigatório	Descrição
<code>-i</code> <code>IMAGE</code>	Sim	Imagem do contêiner de build do cliente (ex.: <code>aws/codebuild/standard:5.0</code>)
<code>-a DIR</code>	Sim	Diretório de saída dos artefatos
<code>-b FILE</code>	Não	Arquivo de substituição do buildspec. Padrão: <code>buildspec.yml</code> no diretório de origem
<code>-s DIR</code>	Não	Diretório de origem. O primeiro <code>-s</code> é a fonte primária; flags <code>-s</code> adicionais usam o formato <code><sourceIdentifier>: <sourceLocation></code> para fontes secundárias. Padrão: diretório de trabalho atual
<code>-l</code> <code>IMAGE</code>	Não	Substitui a imagem padrão do agente local
<code>-r DIR</code>	Não	Diretório de saída dos relatórios
<code>-c</code>	Não	Usa a configuração e credenciais AWS do host local (<code>~/.aws</code> e variáveis de ambiente <code>AWS_*</code>)
<code>-p</code> <code>PROFILE</code>	Não	Perfil da AWS CLI a ser usado (requer <code>-c</code>)
<code>-e FILE</code>	Não	Arquivo contendo variáveis de ambiente (formato <code>VAR=VAL</code> , uma por linha)
<code>-m</code>	Não	Monta o diretório de origem diretamente no contêiner de build
<code>-d</code>	Não	Executa o contêiner de build em modo privilegiado do Docker

Scanners de Segurança

O workflow `security-scanners.yml` executa seis scanners a cada push para `main`, a cada PR direcionado a `main` e em um agendamento diário. Cada scanner envia um relatório

SARIF para o GitHub Code Scanning (visível na aba **Security**) e como um artefato para download.

Todos os scanners, exceto o ClamAV, utilizam um **padrão de falha adiada**: a varredura sempre é executada até o final e os resultados são enviados antes de o job falhar. Isso garante que os achados sejam registrados mesmo quando o build é interrompido.

Bandit — SAST para Python

O que detecta: Problemas comuns de segurança em código Python (ex.: uso de `subprocess`, `eval`, senhas fixas no código, criptografia fraca).

O que causa falha: Qualquer achado com **alta confiança**, em qualquer nível de severidade. Veja a configuração do Bandit em `.github/workflows/security-scanners.yml` para os filtros exatos utilizados.

Escopo: Executa contra todos os arquivos Python rastreados no repositório; veja `.github/workflows/security-scanners.yml` para os padrões precisos de inclusão/exclusão.

Como revisar os achados:

1. Verifique os alertas de **Code Scanning** na aba Security do GitHub, ou baixe o artefato `bandit.sarif`
2. Cada achado inclui um ID de regra do Bandit (ex.: `B603`) e uma descrição do risco

Como corrigir:

- **Corrija o código** — a abordagem preferida. A documentação do Bandit lista alternativas seguras para cada regra
- **Supressão inline** — adicione `# nosec BXXX` (com uma justificativa) à linha afetada:

```
python subprocess.run(cmd, check=True) # nosec B603 - cmd is built from validated config, not user input
```

- **Excluir um caminho** — adicione à lista `exclude` no arquivo `.bandit`

Semgrep — SAST Multilinguagem

O que detecta: Padrões de segurança problemáticos, uso de APIs perigosas e problemas de qualidade de código em todas as linguagens, usando o Semgrep Registry completo (`-config=r/all`).

O que causa falha: Qualquer achado. Em PRs, apenas achados **novos** (em relação ao commit base do PR) causam falha — achados pré-existentes são ignorados via `--baseline-commit`.

Como revisar os achados:

1. Verifique os alertas de **Code Scanning** ou baixe o artefato `semgrep.sarif`
2. Cada achado inclui um ID de regra (ex.: `python.lang.security.dangerous-subprocess-use-audit`) e um link para a documentação da regra

Como corrigir:

- **Corrija o código** — siga a correção sugerida pela regra na documentação do Semgrep Registry
- **Supressão inline** — adicione `# nosemgrep: <rule-id>` à linha afetada:

```
python time.sleep(5) # nosemgrep: arbitrary-sleep - polling for server startup
```

Para arquivos YAML:

```
yaml run: exit ${ steps.scan.outputs.exit_code } # nosemgrep: yaml.github-actions.security.curl-eval.curl-eval
```

- **Excluir um caminho** — adicione o caminho ao `.semgrepignore` (note: a regra de auditoria `changed-semgrepignore` sinalizará novas entradas para revisão de segurança)

Grype — Varredura de Vulnerabilidades em Dependências (SCA)

O que detecta: CVEs conhecidas nas dependências do projeto, varrendo arquivos de lock, manifestos e imagens de contêiner.

O que causa falha: Qualquer vulnerabilidade classificada como **alta ou crítica** (`fail-on-severity: high` no `.grype.yaml`). Vulnerabilidades baixas e médias são reportadas, mas não causam falha no build.

Como revisar os achados:

1. Verifique os alertas de **Code Scanning** ou baixe o artefato `grype.sarif`
2. Cada achado inclui o ID do CVE, o pacote afetado, a versão instalada e a versão corrigida (quando disponível)

Como corrigir:

- **Atualize a dependência** — a abordagem preferida. Verifique se existe uma versão com a correção e atualize o `pyproject.toml` ou o arquivo de lock correspondente
- **Supressão na configuração** — adicione uma entrada à lista `ignore` no `.grype.yaml` com uma justificativa:

```
yml ignore: - vulnerability: CVE-2024-12345 reason: "only affects server-side XML parsing which we don't use"
```

Você pode limitar ao escopo de um pacote específico:

```
yml ignore: - vulnerability: CVE-2024-12345 package: name: "some-package" version: "1.2.3" reason: "pinned version; affected code path is unreachable"
```

Nota: O Grype é um scanner SCA — ele analisa dependências, não linhas de código-fonte. Não há comentários inline para supressão; todos os riscos aceitos devem ser registrados no `.grype.yaml`.

Gitleaks — Detecção de Segredos

O que detecta: Segredos (chaves de API, tokens, senhas, chaves privadas) enviados em qualquer parte do histórico git.

O que causa falha: Qualquer segredo que não esteja presente no arquivo de baseline (`.gitleaks-baseline.json`).

Como revisar os achados:

1. Baixe o artefato `gitleaks.sarif`
2. Cada achado identifica o tipo de segredo (ex.: `generic-api-key`, `jwt`), o arquivo e o commit

Como corrigir:

- **Rotacione o segredo imediatamente** — trate qualquer segredo detectado como comprometido
- **Remova do histórico** — use `git filter-repo` ou BFG Repo-Cleaner para purgar o segredo de todos os commits
- **Adicione ao baseline** — somente para falsos positivos conhecidos (ex.: fixtures de teste com credenciais sintéticas). Regenere o baseline:

```
bash gitleaks git --config=.gitleaks.toml --report-path=.gitleaks-baseline.json --report-format=json .
```

Revise cuidadosamente o baseline atualizado antes de fazer o commit

- **Allowlist de um caminho** — adicione uma expressão regular ao `.gitleaks.toml` em `[allowlist] paths` para arquivos que intencionalmente contêm padrões semelhantes a segredos (ex.: limpadores de credenciais de teste)

Checkov — Varredura de Infraestrutura como Código

O que detecta: Configurações incorretas em workflows do GitHub Actions e Dockerfiles (ex.: actions sem versão fixada, configurações de segurança ausentes, permissões excessivamente amplas).

Escopo: Varre apenas os frameworks `github_actions` e `dockerfile` (configurado no `.checkov.yaml`).

O que causa falha: Qualquer falha de verificação, exceto as verificações listadas em `skip-check`.

Como revisar os achados:

1. Verifique os alertas de **Code Scanning** ou baixe o artefato `checkov.sarif`
2. Cada achado inclui um ID de verificação (ex.: `CKV_GHA_7`, `CKV_DOCKER_2`) e uma descrição da configuração incorreta

Como corrigir:

- **Corrija a configuração** — siga a documentação do Checkov para o ID de verificação específico
- **Supressão inline** — adicione um comentário acima ou na linha afetada:

Em um Dockerfile:

```
dockerfile # checkov:skip=CKV_DOCKER_2:healthcheck not needed for build-only image FROM python:3.12-slim
```

Em um workflow do GitHub Actions:

```
yaml # checkov:skip=CKV_GHA_7:buildspec-override requires user parameters - uses: aws-actions/aws-codebuild-run-build@v1
```

Múltiplos skips em uma linha:

```
yaml # checkov:skip=CKV_DOCKER_2,CKV_DOCKER_3:reason for both
```

- **Ignorar em todo o repositório** — adicione o ID da verificação à lista `skip-check` no `.checkov.yaml` com um comentário explicando o motivo

ClamAV — Varredura de Malware

O que detecta: Malware, vírus e trojans nos arquivos do repositório usando o banco de assinaturas do ClamAV.

O que causa falha: Qualquer detecção de malware (aprovação/reprovação binária).

Como revisar os achados:

1. Baixe o artefato `clamscan.txt` — ele contém o log completo da varredura com os caminhos de arquivos infectados

Nota: O ClamAV não produz saída SARIF e não se integra ao GitHub Code Scanning. Os resultados estão disponíveis apenas como artefato de log de texto.

Como corrigir:

- **Remova o arquivo infectado** e investigue como ele foi introduzido
- **Verifique a detecção** — falsos positivos são raros, mas possíveis. Confira o nome da assinatura do ClamAV em bancos de dados de FPs conhecidos

Resumo dos Limites de Falha

Scanner	Falha em	Filtro de severidade	Arquivo de configuração
Bandit	Qualquer achado com alta confiança	Todas as severidades	<code>.bandit</code>
Semgrep	Qualquer achado (PRs: apenas novos)	Todas as severidades	<code>.semgrepignore</code>
Grype	CVEs altas ou críticas	Baixo/médio não causam falha	<code>.grype.yaml</code>
Gitleaks	Qualquer segredo fora do baseline	Todos	<code>.gitleaks.toml</code> , <code>.gitleaks-baseline.json</code>
Checkov	Qualquer falha de verificação	Todos (menos os ignorados)	<code>.checkov.yaml</code>
ClamAV	Qualquer detecção de malware	Aprovação/reprovação binária	Nenhum

Resumo dos Métodos de Supressão

Scanner	Comentário inline	Nível de configuração	Baseline/diferencial
Bandit	<code># nosec BXXX</code>	<code>.bandit</code> <code>exclude</code>	—
Semgrep	<code># nosemgrep: rule-id</code>	<code>.semgrepignore</code>	<code>--baseline-commit</code> em PRs
Grype	<i>(não aplicável — SCA)</i>	<code>.grype.yaml</code> <code>ignore</code>	—
Gitleaks	—	<code>.gitleaks.toml</code> <code>allowlist</code>	<code>.gitleaks-baseline.json</code>
Checkov	<code>#</code> <code>checkov:skip=ID:reason</code>	<code>.checkov.yaml</code> <code>skip-check</code>	—
ClamAV	—	—	—

Executando GitHub Actions Localmente

NOTA: Este processo usa a ferramenta `act` e assume acesso a um projeto AWS CodeBuild válido `codebuild-project` em "us-east-1"

```
act --platform ubuntu-latest==self-hosted \  
  --job build \  
  --workflows .github/workflows/codebuild.yml \  
  --env-file .env \  
  --var CODEBUILD_PROJECT_NAME=codebuild-project \  
  --var AWS_REGION=us-east-1 \  
  --var ROLE_DURATION_SECONDS=7200 \  
  --artifact-server-path=$PWD/.codebuild/artifacts \  
  --cache-server-path=$PWD/.codebuild/artifacts \  
  --env ACT_CODEBUILD_DIR=$PWD/.codebuild/downloads \  
  --bind
```